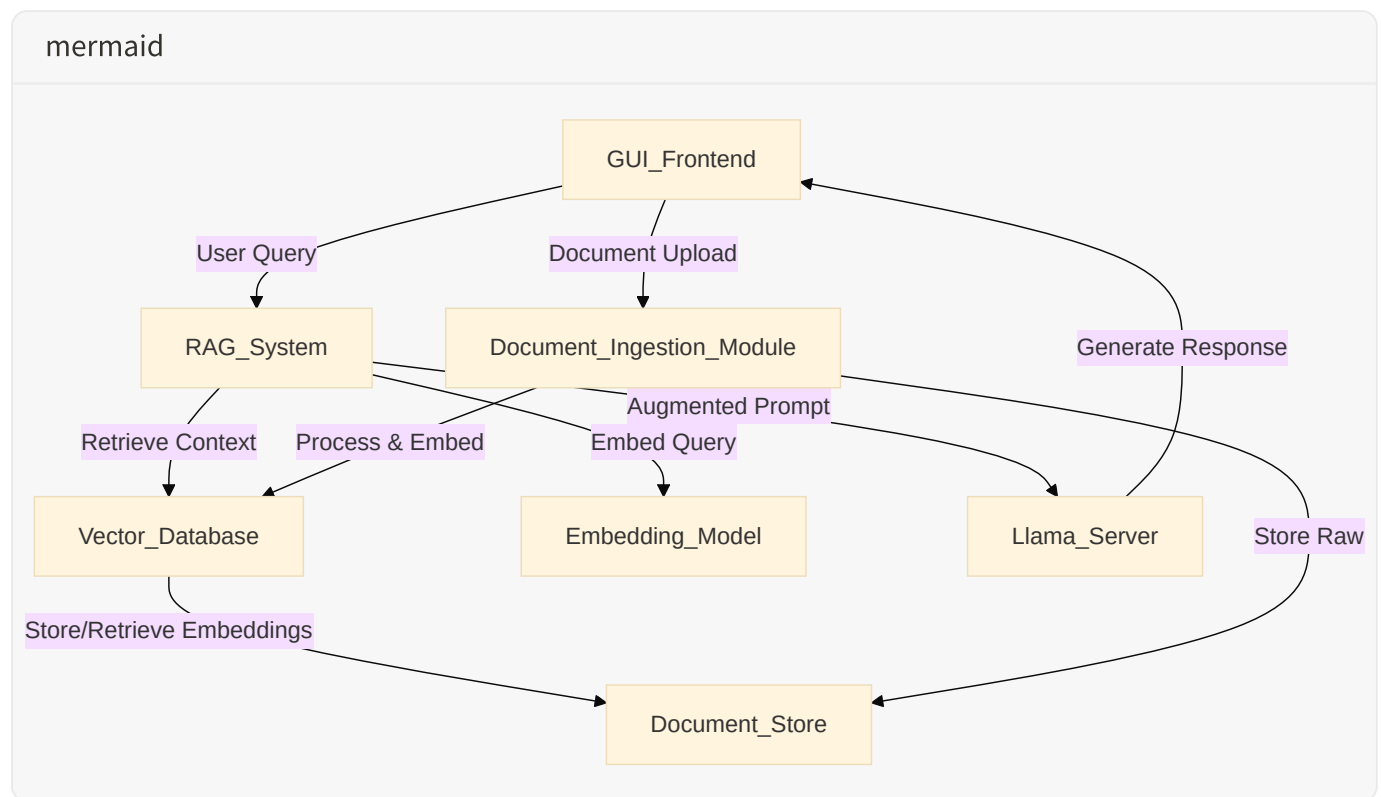


Technical Specification: Retrieval-Augmented Generation (RAG) Implementation

This document details the technical specifications for integrating Retrieval-Augmented Generation (RAG) capabilities into the `llama.cpp` GUI wrapper for Fedora KDE Plasma. The RAG system will enable the LLM to retrieve relevant information from a local knowledge base to enhance its responses, addressing limitations such as knowledge cutoffs and providing more context-aware outputs.

1. System Architecture

The RAG system will operate as a modular component, interacting with the existing `llama-server` and the GUI frontend. It will consist of several key sub-components:



1.1. Components

- **GUI Frontend:** Initiates RAG queries, displays retrieved context, and presents augmented LLM responses. It also provides an interface for document ingestion.
- **RAG System (Backend Module):** Orchestrates the RAG process, including query embedding, retrieval from the vector database, prompt augmentation, and interaction with the `llama-server`.

- **Embedding Model:** A local embedding model (e.g., from llama.cpp or a dedicated huggingface_hub model) responsible for converting text into vector embeddings.
- **Vector Database:** Stores vector embeddings of processed documents, enabling efficient semantic search. For local deployments, options like ChromaDB or LanceDB are suitable [8] [11].
- **Document Store:** Stores the raw text content of ingested documents, linked to their embeddings in the vector database.
- **Document Ingestion Module:** Handles the processing of user-uploaded documents (e.g., PDF, TXT), extracts text, chunks it, generates embeddings, and stores them in the Document Store and Vector Database.
- **llama-server** : The core LLM inference engine, which receives augmented prompts from the RAG system.

2. Data Models

Effective RAG implementation relies on well-defined data models for documents and their associated metadata and embeddings. These models will be used across the Document Store and Vector Database.

2.1. Document Model

This model represents a single logical document or a chunk of a larger document within the RAG system. It will be stored in the Document Store (e.g., a local SQLite database or file system) and referenced by the Vector Database.

Field	Type	Description	Example
id	UUID	Unique identifier for the document chunk.	a1b2c3d4-e5f6-7890-1234-567890abcdef
content	String	The raw text content of the document chunk.	"The quick brown fox jumps over the lazy dog."
source_id	UUID	Reference to the original source document (if chunked).	original_doc_id_123
metadata	JSON	Arbitrary key-value pairs for additional information (e.g., title,	{"title": "Report X", "author": "John Doe"}

		author, date, tags, page number).	
created_at	DateTime	Timestamp when the document was ingested.	2026-07-04T10:30:00Z

2.2. Embedding Model

This model represents the vector embedding of a document chunk, stored in the Vector Database. It will link back to the `Document Model` via the `document_id`.

Field	Type	Description	Example
id	UUID	Unique identifier for the embedding.	f1e2d3c4-b5a6-9876-5432-10fedcba9876
document_id	UUID	Foreign key referencing the <code>id</code> of the associated <code>Document Model</code> .	a1b2c3d4-e5f6-7890-1234-567890abcdef
vector	Array<Float>	The numerical vector representation of the content.	[0.1, 0.5, -0.2, ..., 0.8]
model_id	String	Identifier of the embedding model used to generate this vector.	bge-small-en-v1.5

2.3. Collection Model

This model organizes documents into logical groups, allowing users to manage different knowledge bases. It will be stored in the Document Store.

Field	Type	Description	Example
id	UUID	Unique identifier for the collection.	col_id_abc
name	String	Human-readable name of the collection.	"My Research Papers"

description	String	Optional description of the collection.	"Collection of academic papers on AI."
embedding_model	String	The default embedding model to use for documents in this collection.	bge-small-en-v1.5
created_at	DateTime	Timestamp when the collection was created.	2026-07-04T10:00:00Z

3. API Endpoints and Interaction Flows

The RAG system will expose a set of RESTful API endpoints to facilitate interaction with the GUI frontend and orchestration of the RAG process. These endpoints will be designed for clarity and ease of use, following modern API design principles [4] [6].

3.1. Document Ingestion API

This API will handle the uploading and processing of documents into the RAG system.

Endpoint	Method	Description	Request Body (Example)	Response Body (Example)
/documents/upload	POST	Uploads a document file for processing and ingestion. Supports various formats (PDF, TXT).	<pre>{ "file": <binary_data>, "collection_id": "col_id_abc" }</pre>	<pre>{ "status": "processing", "document_id": "doc_id_xyz" }</pre>
/documents/{id}/status	GET	Retrieves the processing status of an uploaded document.	(None)	<pre>{ "status": "completed", "chunks_processed": 10 }</pre>

3.2. Retrieval API

This API will be used by the GUI to query the RAG system for relevant context based on a user's input.

Endpoint	Method	Description	Request Body (Example)	Response Body (Example)
/retrieve	POST	Retrieves relevant document chunks based on a query.	<pre>{ "query": "What is RAG?", "collection_id": "col_id_abc", "top_k": 5 }</pre>	<pre>{ "retrieved_chunks": [{ "id": "chunk_1", "content": "...", ... }] }</pre>

3.3. Collection Management API

This API will allow users to manage their document collections.

Endpoint	Method	Description	Request Body (Example)	Response Body (Example)
/collections	GET	Lists all available document collections.	(None)	<pre>[{ "id": "col_id_abc", "name": "My Docs", ... }]</pre>
/collections	POST	Creates a new document collection.	<pre>{ "name": "New Collection", "description": "..." }</pre>	<pre>{ "id": "new_col_id", "name": "New Collection" }</pre>
/collections/{id}	DELETE	Deletes a specific document collection and its associated documents.	(None)	<pre>{ "status": "success" }</pre>

3.4. Interaction Flow for RAG Query

- User Input:** The user enters a query into the GUI frontend.
- GUI to RAG System:** The GUI sends the user's query to the /retrieve endpoint of the RAG system.
- RAG System Processing:**
 - The RAG system embeds the user's query using the configured embedding model.
 - It performs a similarity search in the Vector Database to find the top_k most relevant document chunks from the specified collection.
 - The raw content of these chunks is retrieved from the Document Store.

4. **RAG System to llama-server** : The RAG system constructs an augmented prompt by combining the original user query with the retrieved document chunks. This augmented prompt is then sent to the llama-server (e.g., via its /completion or /chat/completions endpoint).
5. **llama-server Response**: The llama-server generates a response based on the augmented prompt.
6. **llama-server to GUI**: The llama-server sends its response back to the GUI frontend.
7. **GUI Display**: The GUI displays the LLM's response to the user, potentially highlighting the retrieved context used.

References

1. [REST API for AI Models Explained: 2026 Guide](#)
2. [AI Agent Memory Systems Compared: RAG vs Local SQLite vs ...](#)
3. [What are the recommendations for choosing a vector database ...](#)
4. [A practical 2026 roadmap for modern AI search & RAG systems](#)
5. [API design best practices guide \(March 2026\)](#)
6. [Top 15 Vector Databases in 2026: A Production Guide](#)
7. [Best open source vector database solutions: Top 5 in 2026](#)
8. [llama.cpp Tutorial: Run a Local LLM in 12 Steps \[2026\]](#)
9. [Implementing RAG, some questions on llama.cpp #12125](#)
10. [Building a RAG Pipeline with llama.cpp in Python](#)